

Machine Learning

Machine Learning is a part of Artificial Intelligence where computers **learn from data** instead of being explicitly programmed.

Example:

- Email spam filter learns which emails are spam
- YouTube recommends videos based on what you watch

How ML Algorithms Work (Step-by-Step)

- **Collect Data:**
Data can be anything: text, images, numbers, etc.
- **Choose an Algorithm:**
Example: Decision Tree, Naive Bayes, etc.
- **Train the Model:**
The algorithm looks at data and finds patterns.
- **Test the Model:**
Check how well it performs on new data.
- **Make Predictions:**
The trained model is used to predict outcomes.

Data → Training → Learning Patterns → Model → Prediction

Features of Machine Learning

1. **Learns from Data:**
No need to manually program rules
2. **Improves Over Time:**
More data → better performance
3. **Automation:**
Reduces human effort
4. **Handles Large Data:**
Works well with big datasets
5. **Prediction Ability:**
Can predict future outcomes
6. **Adaptability:**
Adjusts when new data is given

Types of ML:

- **Supervised Learning** → learns from labelled data
- **Unsupervised Learning** → finds patterns without labels
- **Reinforcement Learning** → learns by trial and error

In **Machine Learning**, algorithms are **methods or techniques** that help computers **learn patterns from data and make decisions**.

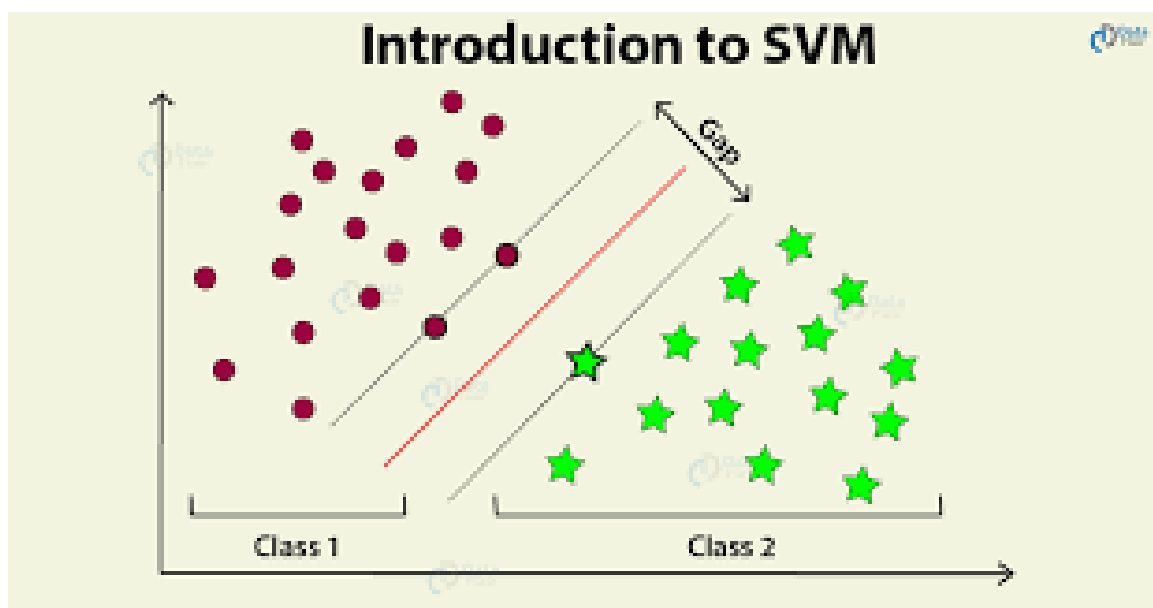
Support Vector Machine (SVM):

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data.

The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.

The closest points to the boundary are called **support vectors**. These support vectors are very important in defining the boundary. SVM works well for both **linear and non-linear data**. For non-linear data, it uses the **Kernel Trick**. It is widely used in tasks like **spam detection, image classification, and text analysis**.



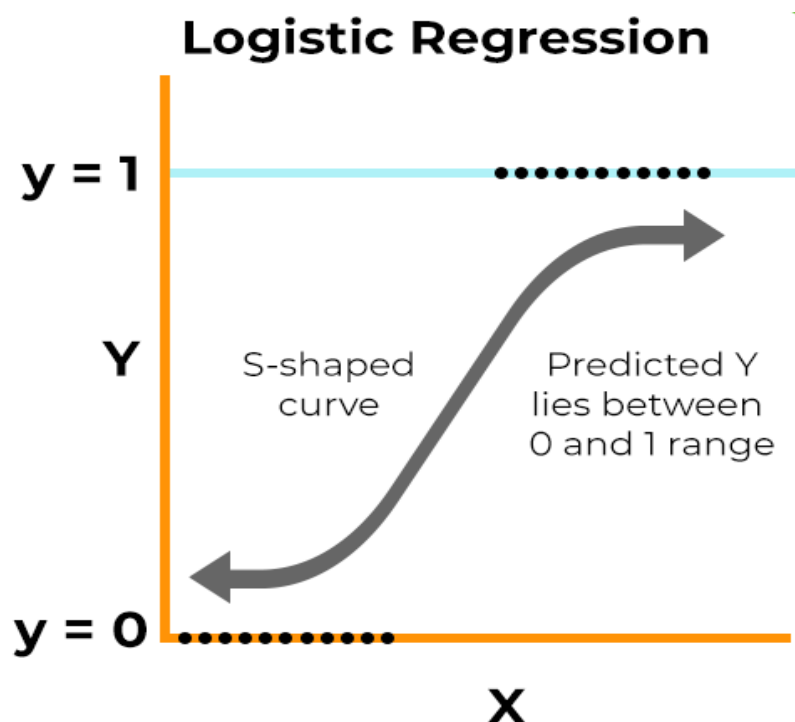
Logistic Regression:

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values it predicts the probability that an input belongs to a specific class.

The core idea of logistic regression is to use a mathematical function called the **Sigmoid Function**, which converts any input value into a range between 0 and 1. This output represents probability. For example, if the result is 0.8, it means there is an 80% chance that the data belongs to a certain class.

Logistic regression works by analyzing the relationship between input features and the output. It assigns weights to each feature and combines them to make predictions. Based on the calculated probability, the model classifies the data into one of the categories using a threshold value.

- It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It uses sigmoid function to convert inputs into a probability value between 0 and 1.



Naive Bayes:

Naive Bayes is a simple and popular algorithm used for classification in Machine Learning. It is based on Bayes' Theorem, which helps to find the probability of a class given some features. The algorithm calculates the probability that a data point belongs to each class. It then selects the class with the highest probability as the final prediction.

The term “naive” means it assumes all features are independent of each other. This assumption makes the calculation simple and fast. In practice, we often use the simplified form $P(C | X) \propto P(C) P(X | C)$. Naive Bayes works well with large datasets and text data. It is commonly used in spam detection, sentiment analysis, and document classification.

Naive Bayes Formula:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Where:

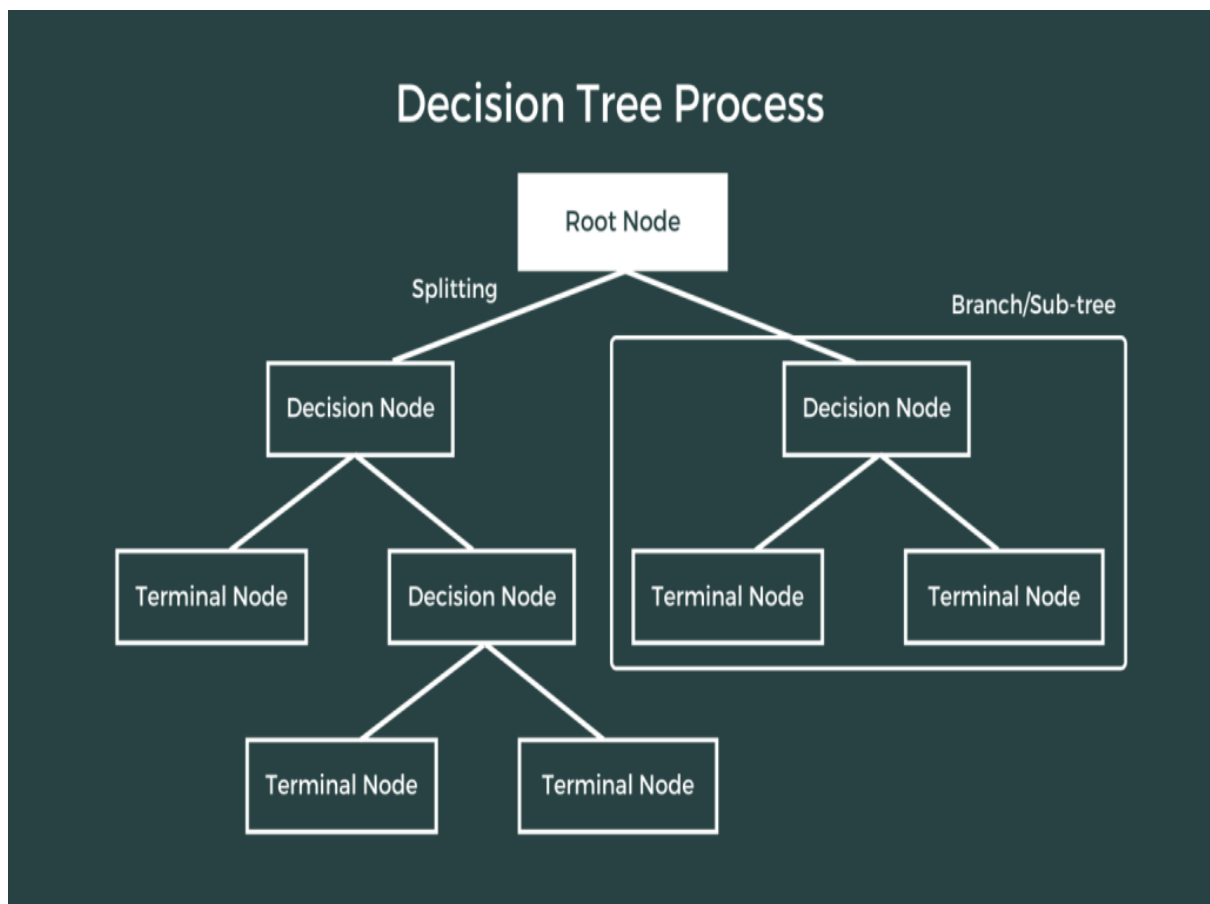
- C= class (e.g., spam or not spam)
- X= feature vector (input data)
- $P(C | X)$ = posterior probability
- $P(X | C)$ = likelihood
- $P(C)$ = prior probability
- $P(X)$ = evidence

Decision Tree Classifier:

Decision Tree is a simple and widely used Machine Learning algorithm for classification and regression. It works like a tree structure, where each node represents a decision based on a feature, and each branch represents the outcome of that decision. The final nodes, called leaf nodes, give the prediction or class label.

The algorithm starts with the entire dataset and selects the best feature to split the data into smaller groups. It keeps splitting the data step by step based on conditions, forming a tree-like structure. This process continues until the data is clearly separated or a stopping condition is reached.

Decision Trees are easy to understand and interpret because they follow a flowchart-like structure. They can handle both numerical and categorical data and require less data preparation. However, they can sometimes overfit the data if the tree becomes too large.



Random Forest Classifier:

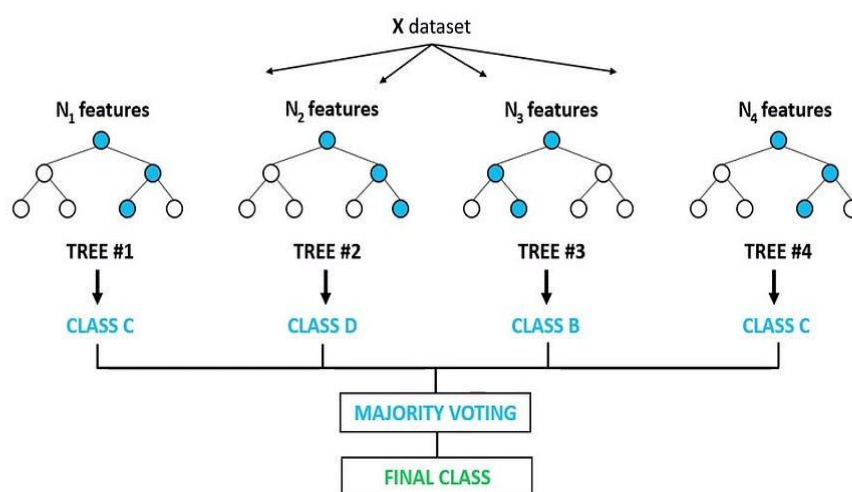
Random Forest is a powerful Machine Learning algorithm used for classification and regression tasks. It is called “random forest” because it builds many decision trees and combines their results to make a final prediction. Instead of relying on a single tree, it uses a group (forest) of trees to improve accuracy.

Each tree in the forest is trained on a random subset of the data and features. This randomness helps create different trees that learn different patterns from the data. When a new input is given, all the trees make their predictions, and the final result is decided by majority voting (for classification) or averaging (for regression).

Random Forest helps to reduce the problem of overfitting that is often seen in a single **Decision Tree**. It provides better accuracy and is more stable. It can also handle large datasets and works well with both numerical and categorical data.

Because of its high performance and reliability, Random Forest is widely used in applications such as fraud detection, medical diagnosis, and recommendation systems.

Random Forest Classifier



Ridge Classifier:

Ridge Classifier is a supervised machine learning algorithm used for classification problems. It is based on linear models and works by finding a straight-line decision boundary between different classes.

Unlike simple linear models, Ridge Classifier uses a technique called **regularization**, which helps reduce overfitting. It does this by adding a penalty to large coefficients, making the model more stable and generalizable to new data.

During training, the algorithm learns weights for each feature and tries to minimize errors while also keeping the weights small. This balance helps improve prediction performance, especially when the dataset has many features or correlated inputs.

Ridge Classifier is fast, simple, and works well for large datasets. It is commonly used in text classification, binary classification, and other problems where linear separation is effective.

Ridge Classifier

What is Ridge Classifier?

Ridge Classifier is a linear classification model that applies L2 regularization (ridge penalty) to the weights to prevent overfitting and handle multicollinearity.

Objective Function

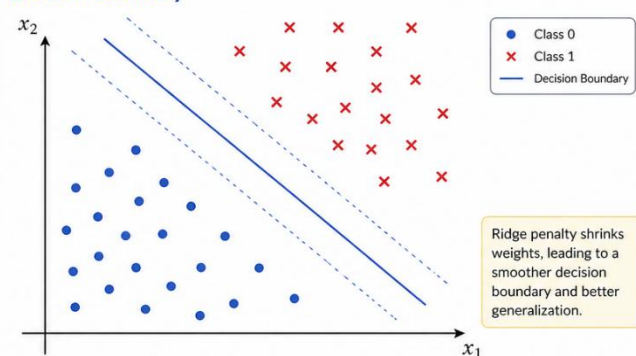
Ridge Classifier minimizes:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \lambda \sum_{j=1}^p w_j^2$$

where:

- $\ell(y_i, \hat{y}_i)$ is the loss function (e.g., logistic loss)
- w is the weight vector
- b is the bias (intercept)
- $\lambda \geq 0$ is the regularization strength (larger $\lambda \Rightarrow$ stronger penalty)
- p is the number of features

Decision Boundary



How it works

- 1 Compute linear scores:

$$z = w^T x + b$$

- 2 Convert to probabilities using sigmoid function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- 3 Predict class:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Ridge vs Logistic Regression

	Logistic Regression	Ridge Classifier
Regularization	None (or L2 on features only)	L2 on weights (ridge penalty)
Objective	Minimize loss	Minimize loss + $\lambda \sum w_j^2$
Effect	May overfit with highly correlated features	Reduces overfitting, more stable coefficients
Equation	$\min \frac{1}{n} \sum \ell(y_i, \hat{y}_i)$	$\min \frac{1}{n} \sum \ell(y_i, \hat{y}_i) + \lambda \sum w_j^2$

Key Takeaway



Ridge Classifier adds an L2 penalty to the weights, shrinking them towards zero and helping the model generalize better, especially when features are correlated or the model is complex.

K-Nearest Neighbors (KNN):

K-Nearest Neighbors (KNN) is a simple and intuitive Machine Learning algorithm used for classification and regression problems. It works on the principle that similar things exist close to each other.

When a new data point is given, KNN finds the “K” closest data points in the training set. It then checks the labels of these nearest points and assigns the most common label to the new data point. For regression problems, it takes the average of the nearest values instead of voting.

The algorithm does not learn a fixed model during training. Instead, it stores all training data and makes decisions during prediction. This is why KNN is also called a “lazy learning” algorithm.

The value of K (number of neighbors) is important because it affects accuracy. A small K makes the model sensitive to noise, while a large K makes it smoother but may reduce detail.

***k*-Nearest Neighbors (KNN)**

1. What is KNN?

K-Nearest Neighbors (KNN) is a simple, non-parametric, instance-based learning algorithm used for classification and regression.

It makes predictions based on the majority class (classification) or average value (regression) of the k closest training examples in the feature space.

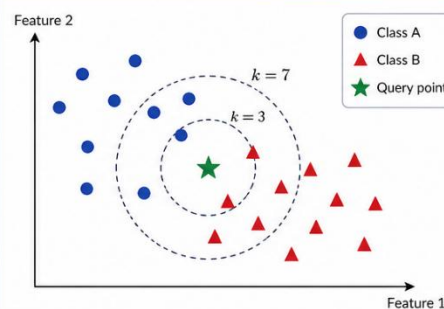
2. How KNN Works (Classification)

- 1 Choose the number of neighbors, k .
- 2 Compute the distance between the query point and all training points.
- 3 Select the k nearest neighbors.
- 4 Assign the class that appears most frequently among those neighbors (majority voting).

Common Distance Metrics

- Euclidean: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Manhattan: $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
- Minkowski (general form):
$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

3. Example (Classification)



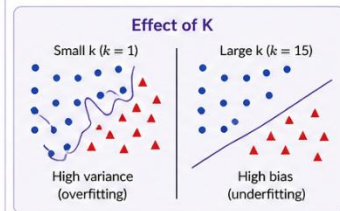
- For $k = 3$: Neighbors $\rightarrow$ 2 blue, 1 red $\Rightarrow$ Predict Class A
 - For $k = 7$: Neighbors $\rightarrow$ 4 blue, 3 red $\Rightarrow$ Predict Class A
- Predicted class: Class A**

KNN: Summary

Aspect	Details
Type	Non-parametric, instance-based (lazy learning)
Used for	Classification, Regression
Training time	Low (just store the data)
Prediction time	Higher (compute distance to all points)
Assumption	Similar points are close in the feature space
Sensitive to	Irrelevant features, scaling, noise
Feature scaling	Important (e.g., use Standardization or Normalization)

4. Choosing K

- Small k (e.g., $k = 1$): sensitive to noise, may overfit.
- Large k : smoother decision boundary, may underfit.
- Use odd k for binary classification to avoid ties.
- Choose k using cross-validation.



5. Key Points

- KNN makes predictions based on the nearest training examples.
- It creates non-linear decision boundaries.
- No training phase; it stores all data.
- Performance depends on the choice of k , distance metric, and feature scaling.